

NED UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Karachi, Pakistan

Course: Network and Information Security

Course Code: CT-486

Complex Computing Problem (CCP) - CLO3

DESIGN AND BREAK A CUSTOM CIPHER BASED ON CLASSICAL TECHNIQUES

HCC - Hybrid Classical Cipher (Vigenere + Rail Fence)

Project Report

Submitted by (Team Members)

S. No.	Name	Roll No.
1	Syeda Aiman Munawar	GA-23003
2	Naverah Farhat	GA-23004
3	Duaa Khalid	GA-23303
4	Zubia Junaid	GA-23307

Course Advisor

Dr. Shariq Mahmood Khan

Table of Contents

Abstract 6
1. Introduction 7
1.1 Problem Statement 7
1.2 Objectives 7
2. Background: The Selected Classical Techniques 7
2.1 Vigenere Cipher (Stage 1) 7
2.2 Rail Fence Cipher (Stage 2) 8
2.3 Why These Two Ciphers 8
3. Design of HCC: Combining the Two Techniques 8
3.1 Architecture 8
3.2 Key Structure and Validation 8
3.3 Mathematical Formulation 9
3.4 How the Two Stages Work Together: A Worked Example 9
4. Encryption and Decryption Algorithms 10
4.1 Encryption Algorithm 10
4.2 Decryption Algorithm 11
4.3 Why Decryption Reverses the Order of the Stages 12
4.4 Complexity of the Algorithms 12
5. Attack Methods and Results 12
5.1 Threat Model: What the Attacker Knows and What Must Be Recovered 12
5.2 Mathematics of the Known-Plaintext Attack 12
5.3 Attack Algorithm 13
5.4 Attack Demonstration with Intermediate Calculations 14
5.5 Frequency Analysis 15
5.6 Bonus: Ciphertext-Only Attack 15
5.7 Experimental Results of the Attacks 16
6. Security Analysis 17
6.1 Benefits of Combining Substitution and Transposition 17
6.2 Strengths and Weaknesses of the Individual Stages 17
6.3 The Effect of Using Both Together 18
6.4 Vulnerability to Known-Plaintext Attacks 18
6.5 Vulnerability to Frequency Analysis 18

6.6 Why HCC Is Educational and Not Suitable for Real Security 18
7. Performance Metrics 18
7.1 Measurement Method 18
7.2 Encryption, Decryption and Attack Time 19
7.3 Comparison with the Shift Cipher 19
7.4 Time and Space Complexity of All Major Algorithms 20
8. Suggested Improvements 20
9. Implementation and Source Code 21
9.1 Running the Project 21
9.2 Testing 21
9.3 Source Code Repository 22
10. Limitations 22
11. Conclusion 22
References 22

List of Figures

Figure 1: HCC architecture: encryption pipeline and the reverse decryption pipeline.....8
Figure 2: Stage 2 on the example: zig-zag layout and the resulting ciphertext.....10
Figure 3: Flowchart of the HCC encryption algorithm.....11
Figure 4: Flowchart of the HCC decryption algorithm.....11
Figure 5: Flowchart of the known-plaintext attack.....13
Figure 6: Letter frequencies: English plaintext, Shift cipher and HCC ciphertext (dashed line = standard English).....15
Figure 7: Known-plaintext attack success rate against the known-plaintext length k.....16
Figure 8: Mean running time against message length (log-log scale): cipher operations (left) and the known-plaintext attack (right).....19

List of Tables

Table 1: Stage 1 (Vigenere) calculation for the first six letters 9
Table 2: Threat model for the known-plaintext attack 12
Table 3: Search log: key derived from the known segment for each rail guess 14
Table 4: Derivation of key letters for the correct hypothesis (r = 4); selected positions 14
Table 5: Known-plaintext attack: success rate and effort (m = 12, rails 2-20, 200 trials per row) 16
Table 6: Ciphertext-only attack: success rate against ciphertext length (40 trials per row, m = 12) 17
Table 7: Strengths and weaknesses of the two stages 17
Table 8: Measured mean execution time in milliseconds for HCC, Shift cipher and the known-plaintext attack 19
Table 9: HCC compared with the single-stage Shift cipher 19
Table 10: Complexity summary 20
Table 11: Improvements, their security benefit and their added cost 20
Table 12: Project structure 21

Abstract

This report presents **HCC (Hybrid Classical Cipher)**, a two-stage cipher built from two classical techniques: a Vigenere substitution with a 12-letter keyword (Stage 1) followed by a Rail Fence transposition with a configurable number of rails (Stage 2). The cipher is implemented in Python with strict input validation, a menu-driven command-line interface and 52 automated tests. A realistic **known-plaintext attack** is implemented that is never given the secret keys: it searches the rail count, undoes the transposition for every guess, derives the Vigenere key letters by modular subtraction and accepts a guess only if the letters form a consistent repeating keyword. Over 200 random trials per setting, the attack recovered the keys and the full plaintext in 100% of trials whenever the known segment was at least one letter longer than the key and the English-likeness tie-break was used, at a cost of about 19 x n character operations (milliseconds). A frequency-analysis module shows why a plain histogram attack fails on HCC, and a bonus ciphertext-only attack shows that the cipher still falls to column-wise statistics given about 500 letters. Encryption and decryption are O(n) and measured 2 to 3.7 times slower than a single-stage Shift cipher. The report concludes that HCC is a useful teaching design but is not secure against modern cryptanalysis, and proposes inexpensive improvements.

1. Introduction

Before computers, secret writing relied on two elementary operations: **substitution**, which replaces letters by other letters, and **transposition**, which rearranges the order of letters. Shannon later described the same two ideas as confusion and diffusion, and modern block ciphers still combine them over many rounds [1], [2]. This project explores the idea in its simplest form: we design a custom cipher from two classical techniques, implement it, and then, as the assignment requires, try to break it.

1.1 Problem Statement

Design a custom cipher that combines at least two different classical encryption techniques in at least two stages, with a key of at least 10 characters. Implement encryption and decryption for messages of different lengths made of the letters A-Z. Implement an attack using frequency analysis and/or known plaintext, document its success rate and computational effort for different ciphertext lengths, analyse encryption and decryption complexity, compare with a single-stage cipher such as the Shift cipher, and suggest security improvements that do not greatly increase complexity.

1.2 Objectives

- Design and justify a two-stage cipher (substitution followed by transposition).
- Implement correct, validated encryption and decryption and test them thoroughly.
- Implement a known-plaintext attack that genuinely recovers the keys from limited attacker knowledge.
- Implement frequency analysis and explain why it alone does not break Vigenere-based output.
- Measure (not assume) running times, attack success rates and attack effort.
- Analyse complexity, compare with the Shift cipher, state limitations and propose improvements.

2. Background: The Selected Classical Techniques

Classical ciphers work directly on letters using rules that can be applied by hand. They fall into families: monoalphabetic substitution (Shift/Caesar, where one fixed mapping is used and letter frequencies survive), polyalphabetic substitution (Vigenere, where several mappings are used in rotation) and transposition (Rail Fence, where letters keep their identity but change position). Kerckhoffs' principle states that a cipher should remain secure even when everything except the key is public [6]. All attacks in this report therefore assume that the attacker knows the algorithm.

2.1 Vigenere Cipher (Stage 1)

Letters are mapped to numbers A = 0, B = 1, ..., Z = 25. For a keyword K of length m, the i-th plaintext letter is shifted by the i-th key letter, repeating the keyword as needed:

$$C_i = (P_i + K_{i \bmod m}) \bmod 26 \qquad P_i = (C_i - K_{i \bmod m}) \bmod 26$$

Strength: the same plaintext letter is encrypted with up to m different shifts, so the ciphertext histogram is much flatter than English, and the key space is 26^m. **Weakness:** the key repeats, so every m-th letter is encrypted with the same Shift cipher. Kasiski examination and the index of coincidence reveal m, and each column can then be solved by frequency analysis [3], [5].

2.2 Rail Fence Cipher (Stage 2)

The text is written diagonally on r rows ("rails"), moving down and then up, and is then read row by row (Figure 2). It is a fixed permutation of positions that depends only on the message length n and the rail count r. **Strength:** it destroys the positional structure that Kasiski and IC key-length attacks rely on. **Weakness:** letters are not changed, so letter frequencies are preserved, and there are only about n - 1 possible keys, which is trivially searched.

2.3 Why These Two Ciphers

They are the standard textbook representatives of the two families, are easy to implement and to verify by hand, and are complementary: the substitution stage hides *which* letters occur, the transposition stage hides *where* the key repeats. Their different weaknesses also make the attack design interesting.

3. Design of HCC: Combining the Two Techniques

3.1 Architecture

HCC applies the two techniques in sequence, so that the output of Stage 1 becomes the input of Stage 2 (Figure 1). The secret key is the pair **(keyword K, rails r)**.

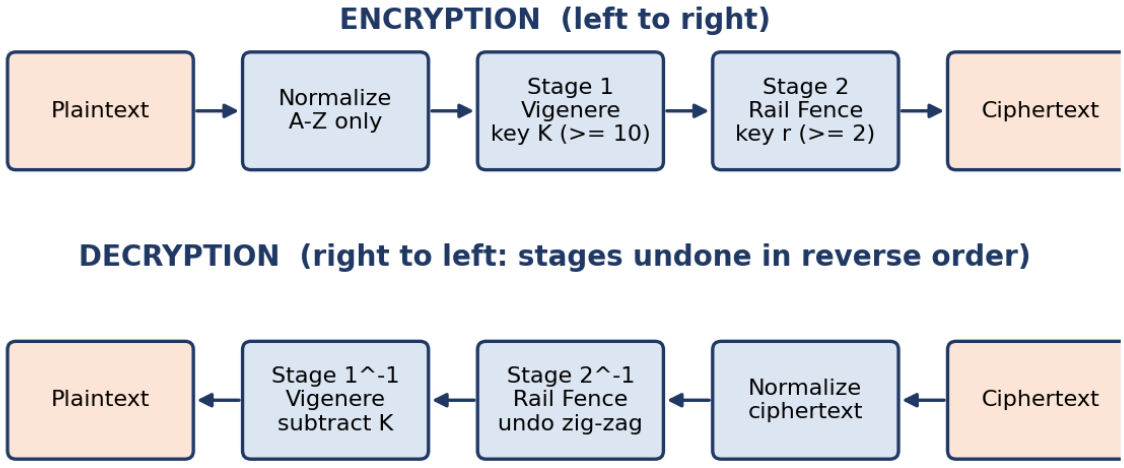


Figure 1: HCC architecture: encryption pipeline and the reverse decryption pipeline

3.2 Key Structure and Validation

- **Stage 1 key:** a keyword of letters A-Z, at least 10 characters (the demonstration key `SECURITYKEYS` has 12).

- **Stage 2 key:** an integer number of rails r >= 2, chosen by the user.
- **Weak-key rule:** a keyword that is just a shorter word repeated (for example `ABCABCABCABC`, effective length 3, or `AAAAAAAAAAAA`) is rejected, because its effective length would be below the 10-character requirement.
- **Short messages:** if r is at least the message length, the zig-zag has one letter per rail and Stage 2 does nothing; this is allowed but a warning is shown.
- **Input normalization:** text is converted to upper case and every character that is not A-Z (spaces, digits, punctuation, accented letters) is removed. Formatting is deliberately not restored because word lengths and punctuation would leak structure to an attacker. Decryption therefore returns the normalized plaintext.

3.3 Mathematical Formulation

Let $P = (p_0, ..., p_{n-1})$ with p_i in Z_{26} and $K = (k_0, ..., k_{m-1})$. Let pi_r be the Rail Fence permutation: the j-th letter read off is the letter at position $pi_r(j)$ of the input. With $L = 2(r - 1)$ and $t = i \bmod L$, position i lies on rail t if $t < r$ and on rail $L - t$ otherwise; pi_r lists the positions sorted by (rail, position).

$$\textit{Stage 1:}\quad v_i = (p_i + k_{i \bmod m}) \bmod 26$$

$$\textit{Stage 2:}\quad c_j = v_{pi(j)}$$

$$\textit{Decrypt:}\quad v_{pi(j)} = c_j \quad \textit{then} \quad p_i = (v_i - k_{i \bmod m}) \bmod 26$$

Both stages are bijections, so each can be undone; the composition of the two inverses applied in reverse order is the exact inverse of the cipher. The key space is 26^{12} = 95,428,956,661,682,176 (about $2^{56.4}$) keywords multiplied by roughly n possible rail counts.

3.4 How the Two Stages Work Together: A Worked Example

We encrypt `MEETMEATTHELIBRARYTOMORROW` (n = 26) with keyword `SECURITYKEYS` (m = 12) and r = 4.

Stage 1: Vigenere substitution

Table 1: Stage 1 (Vigenere) calculation for the first six letters

i	Plaintext p_i	Key letter k_(i mod 12)	p_i + k_i	Stage 1 output v_i
0	M (12)	S (18)	30 mod 26 = 4	E (4)
1	E (4)	E (4)	8	I (8)
2	E (4)	C (2)	6	G (6)
3	T (19)	U (20)	39 mod 26 = 13	N (13)
4	M (12)	R (17)	29 mod 26 = 3	D (3)
5	E (4)	I (8)	12	M (12)

Complete Stage 1 output: `EIGNDMTRDLCDAFTUIGMMWSPJGA`.

Stage 2: Rail Fence transposition

The Stage 1 output is written on 4 rails and read rail by rail (Figure 2).

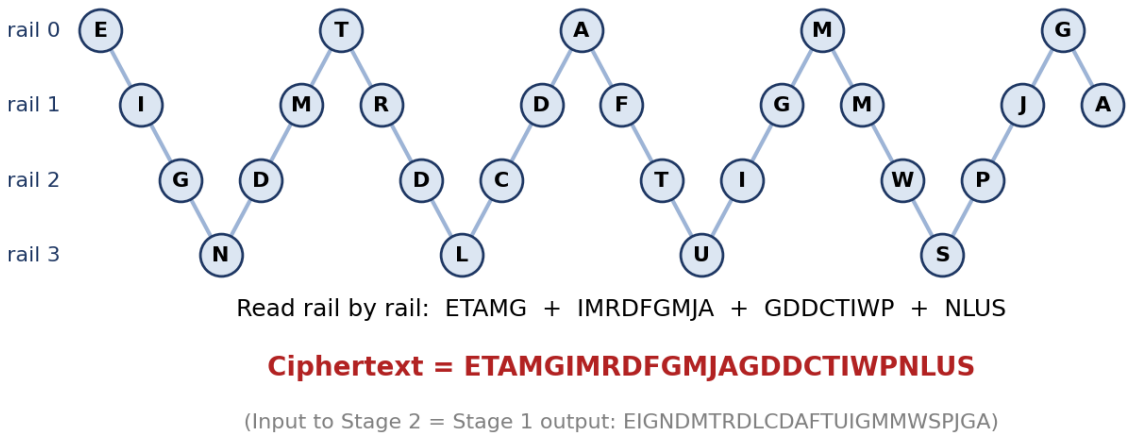


Figure 2: Stage 2 on the example: zig-zag layout and the resulting ciphertext

Why the combination is stronger than either stage alone against simple attacks: after Stage 1, the letter statistics are flattened, so a plain frequency count no longer matches one shift. After Stage 2, the letters that share a key letter are no longer at regular intervals, so the Kasiski/IC technique for finding the key length and splitting the text into Shift-cipher columns no longer applies directly. The attacker must handle both problems at once. **Why it is still weak:** the two stages are separable, which the known-plaintext attack in Section 5 exploits.

4. Encryption and Decryption Algorithms

4.1 Encryption Algorithm

```
ALGORITHM HCC-Encrypt(message, keyword, rails)
  1. keyword <- validate(keyword)       # A-Z only, length >= 10, not a repeated shorter
word
  2. rails   <- validate(rails)         # integer >= 2
  3. P <- uppercase(message) with every non A-Z character removed
     IF P is empty THEN error
  4. FOR i = 0 TO n-1:                  # Stage 1: Vigenere
         V[i] <- (P[i] + K[i mod m]) mod 26
  5. FOR i = 0 TO n-1:                  # Stage 2: Rail Fence
         rail[i] <- zigzag(i, rails)    # 0,1,..,r-1,r-2,..,1,0,1,...
  6. C <- letters of V grouped by rail 0, then rail 1, ... (left to right inside a rail)
  7. RETURN C
```

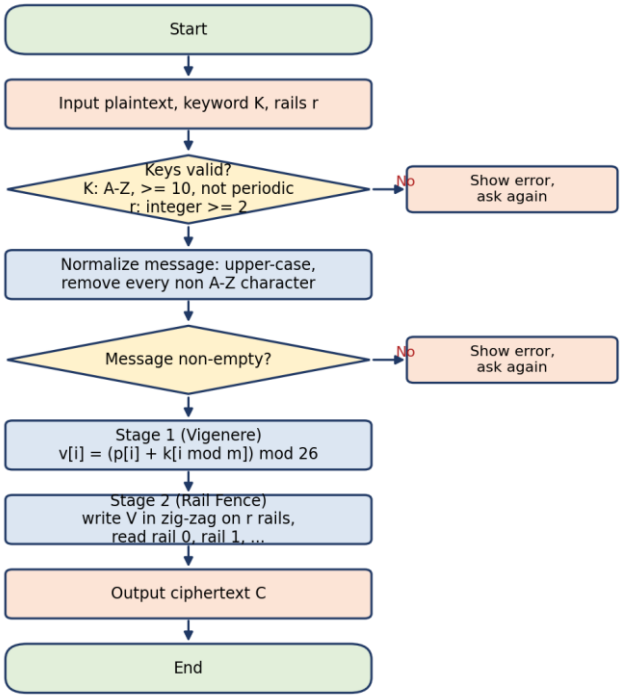


Figure 3: Flowchart of the HCC encryption algorithm

4.2 Decryption Algorithm

```
ALGORITHM HCC-Decrypt(ciphertext, keyword, rails)
  1. validate keyword and rails; C <- normalize(ciphertext)
  2. order <- the positions 0..n-1 sorted by (zigzag rail, position)   # same
permutation as encryption
  3. FOR j = 0 TO n-1:                    # undo Stage 2
        V[order[j]] <- C[j]
  4. FOR i = 0 TO n-1:                    # undo Stage 1
        P[i] <- (V[i] - K[i mod m]) mod 26
  5. RETURN P
```

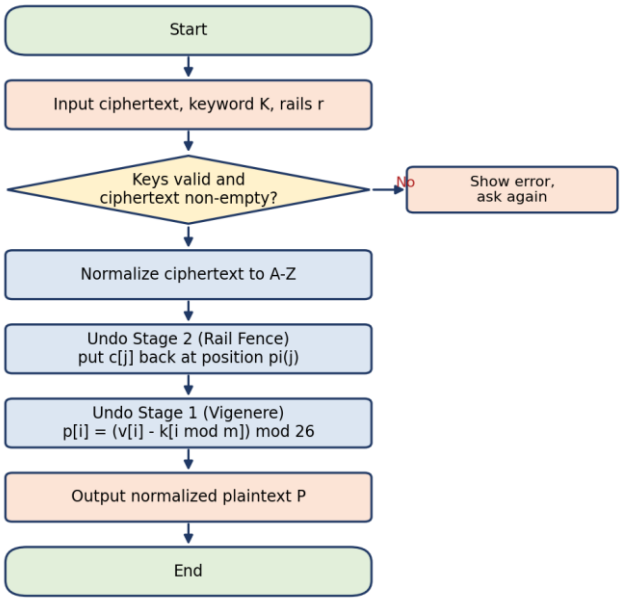


Figure 4: Flowchart of the HCC decryption algorithm

4.3 Why Decryption Reverses the Order of the Stages

Encryption is the composition RailFence(Vigenere(P)). The inverse of a composition is the composition of the inverses in reverse order: $\text{Vigenere}^{-1}(\text{RailFence}^{-1}(C))$. If Vigenere were undone first, key letters would be subtracted from letters that are no longer at their original positions and the result would be unreadable. A unit test confirms that applying the two stages in the wrong order gives a different ciphertext.

4.4 Complexity of the Algorithms

Vigenere performs one modular operation per letter. The Rail Fence permutation is built in one pass over the message by placing each position into a bucket for its rail, which costs O(n) regardless of r. Hence encryption and decryption of HCC are each **O(n) time and O(n) space**. The measurements in Section 7 confirm linear growth.

5. Attack Methods and Results

5.1 Threat Model: What the Attacker Knows and What Must Be Recovered

Table 2: Threat model for the known-plaintext attack

Assumed known to the attacker	Must be recovered by the attack
Algorithm: Vigenere first, Rail Fence second	Rail count r
The complete ciphertext C (length n)	Keyword K (m letters)
A known plaintext segment of k letters and its position s in the message	The rest of the plaintext
The keyword length m (or a small range of candidates)	
A range of possible rail counts [r_min, r_max]	

The attack function takes only these inputs; there is no parameter for the secret keys, and a unit test verifies the function signature. The known segment is contiguous in the *plaintext* but is scattered over the *ciphertext* after the transposition, so the attacker needs the whole ciphertext.

5.2 Mathematics of the Known-Plaintext Attack

For a guessed rail count r the attacker applies the inverse permutation to the ciphertext, obtaining V'. If r is correct then V' = V, the Stage 1 output. For every known position $i = s + t$, Stage 1 gives $v_i = p_i + k_{i \bmod m}$, so the key letter in slot i mod m is:

$$k'_{i \bmod m} = (V'_i - P_i) \bmod 26$$

Positions with the same residue i mod m must produce the same key letter. For the correct r this holds for every pair, so the derived key is a **consistent repeating keyword**. For a wrong r, V' is a scrambled copy of V and two positions agree only by chance (probability about 1/26 each).

Let rho = k - min(k, m) be the number of redundant equations (known letters beyond one per key slot). A wrong rail count survives the consistency test with probability of about $26^{-\text{rho}}$. With R candidate rail counts the expected number of surviving hypotheses is $1 + (R - 1) \times 26^{-}$

rho. For R = 19: rho = 0 gives 19 survivors (no decision possible), rho = 1 gives 1.69 and rho = 2 gives 1.03. Consequently **k must be greater than m** for a reliable result, and **k >= m** is needed to recover the whole keyword at all.

5.3 Attack Algorithm

```
ALGORITHM KnownPlaintextAttack(C, P_known, M, [r_min, r_max], s)
  FOR r = r_min TO r_max:                                  # step 1: guess the rail count
      V' <- RailFence_r^-1(C)                              # step 2: undo the transposition
O(n)
      FOR EACH assumed key length m IN M:
          slots[0..m-1] <- empty ; conflicts <- 0
          FOR t = 0 TO k-1:
              i <- s + t
              d <- (V'[i] - P_known[t]) mod 26             # step 3: key letter by modular
arithmetic
              IF slots[i mod m] is empty: slots[i mod m] <- d
              ELSE IF slots[i mod m] != d: conflicts <- conflicts + 1   # step 4:
consistency check
          IF conflicts = 0: add (r, m, slots) to SURVIVORS
  IF SURVIVORS is empty: report FAILURE
  rank SURVIVORS: complete keys first, then by English-likeness of the full decryption
  best <- first survivor                                   # step 5: identify the valid
candidate
  IF best has unknown key slots: report PARTIAL (known segment too short)
  P <- Vigenere^-1(RailFence_best.r^-1(C), best.key)   # step 6: decrypt everything
  verify HCC-Encrypt(P, best.key, best.r) = C; compare with the original   # step 7
```

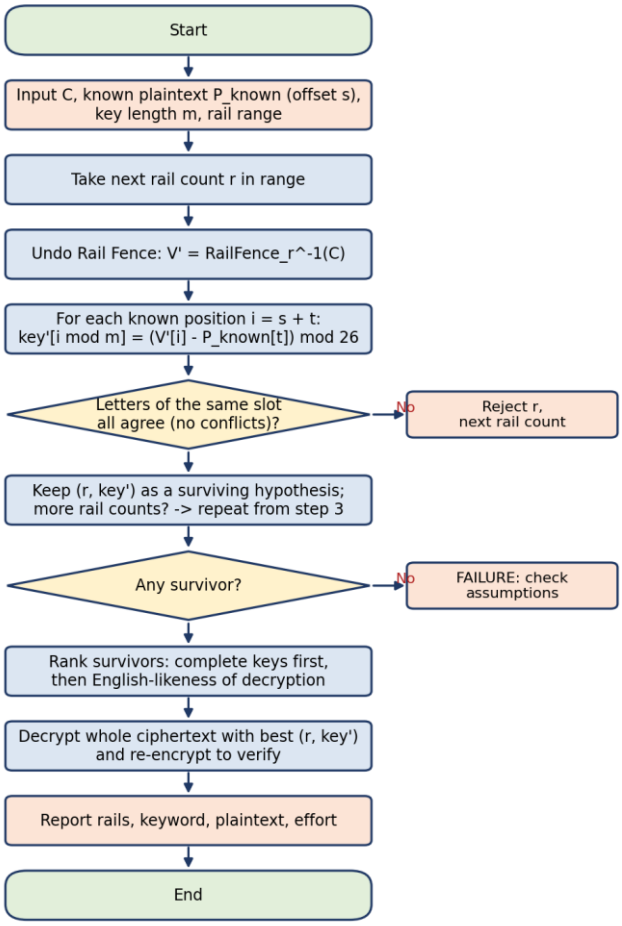


Figure 5: Flowchart of the known-plaintext attack

The English-likeness ranking uses the chi-squared distance between the decrypted text and standard English letter frequencies. It is only needed when several hypotheses survive, and it is an additional assumption (the plaintext is English); results are therefore reported both with it ("ranked") and without it ("strict": exactly one hypothesis survived and it was correct).

5.4 Attack Demonstration with Intermediate Calculations

The attacker knows the ciphertext `ETAMGIMRDFGMJAGDDCTIWPNLUS`, the first k = 16 plaintext letters `MEETMEATTHELIBRA` (offset 0), m = 12 and a rail range of 2 to 8. The table below is the real output of the attack function.

Table 3: Search log: key derived from the known segment for each rail guess

Rail guess r	Derived key k'	Conflicts (of 4 redundant letters)	Verdict
2	`SWPNOZMKNVEI`	4	rejected
3	`SNSKHBPNHFJY`	4	rejected
4	`SECURITYKEYS`	0	**consistent (accepted)**
5	`SCIJZPJPAFWX`	4	rejected
6	`SIZHHHINMZPX`	4	rejected
7	`SIZQRSUWJTBV`	4	rejected
8	`SWEKXZWBWVWU`	4	rejected

For r = 4 the key letter at each known position is (V' - P) mod 26. Positions 12 to 15 fall into slots 0 to 3 again and agree with the letters found earlier, which proves consistency:

Table 4: Derivation of key letters for the correct hypothesis (r = 4); selected positions

Index i	P_i	V'_i	(V' - P) mod 26	Key letter	Slot i mod 12
0	M	E	(4 - 12) mod 26 = 18	S	0
1	E	I	(8 - 4) mod 26 = 4	E	1
2	E	G	(6 - 4) mod 26 = 2	C	2
3	T	N	(13 - 19) mod 26 = 20	U	3
11	L	D	(3 - 11) mod 26 = 18	S	11
12	I	A	(0 - 8) mod 26 = 18	S	0
13	B	F	(5 - 1) mod 26 = 4	E	1
14	R	T	(19 - 17) mod 26 = 2	C	2
15	A	U	(20 - 0) mod 26 = 20	U	3

Result: rails = 4, keyword = `SECURITYKEYS`. Decrypting the whole ciphertext gives `MEETMEATTHELIBRARYTOMORROW`, which equals the original message, and the recovered keys re-encrypt it to the supplied ciphertext. The attack tested 7 hypotheses with 346 character operations.

5.5 Frequency Analysis

The frequency-analysis module counts letters, compares them with standard English frequencies [4] using the chi-squared statistic and computes the index of coincidence (IC), the probability that two randomly chosen letters of a text are equal: IC = sum $n_i(n_i - 1) / (N(N - 1))$. English text has IC of about 0.0655 and uniformly random text about 0.0385 [5]. Figure 6 compares a 1000-letter English sample, its Shift-cipher encryption and its HCC encryption.

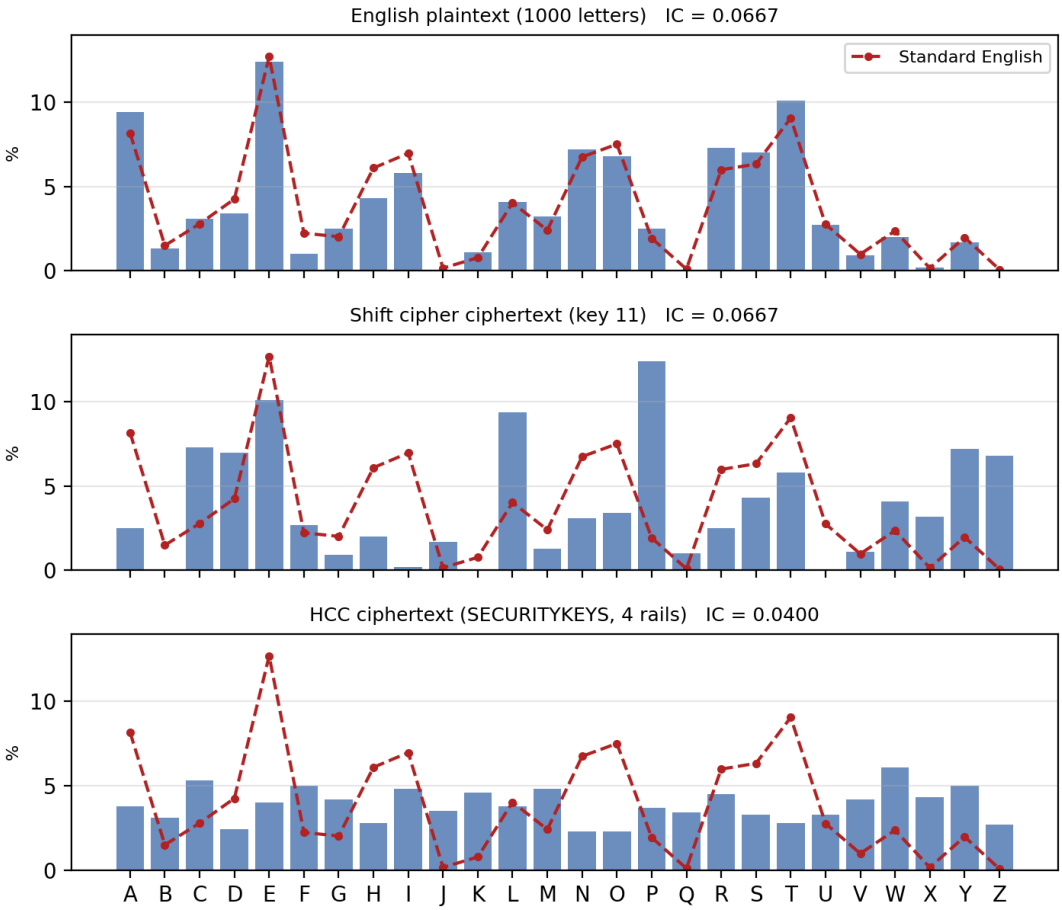


Figure 6: Letter frequencies: English plaintext, Shift cipher and HCC ciphertext (dashed line = standard English)

Why frequency analysis alone fails against HCC. (1) A Shift cipher maps each letter to exactly one other letter, so the histogram is only rotated and 26 trials identify the key; the program recovers the key this way. (2) Vigenere maps a plaintext letter to up to m different ciphertext letters, so the histogram flattens (IC 0.0400 for HCC compared with 0.0667 for the English sample) and no single shift makes it resemble English. (3) Rail Fence does not change the histogram but hides the positions (i mod m) needed to split the text into Shift-cipher columns. **Caveat:** this defeats only the naive attack, as the next section shows.

5.6 Bonus: Ciphertext-Only Attack

For each rail count the attack undoes the transposition, splits the result into m columns (each column is a Shift cipher), chooses for each column the shift with the smallest chi-squared distance from English, and adds the column scores; the rail count with the lowest total wins. It uses no known plaintext.

5.7 Experimental Results of the Attacks

Test conditions. 200 trials per row; each trial uses a fresh random non-periodic 12-letter keyword, a random rail count in [2, 20], and an English-like plaintext made of randomly chosen sentences from a built-in corpus; the known segment is the first k letters; the attacker is told m = 12 and the rail range 2 to 20. Random generators are seeded, so the experiment is reproducible (`performance.py`). *Strict* = exactly one hypothesis survived and it was correct. *Ranked* = final answer correct after English-likeness ranking. *Char ops* = elementary letter operations performed by the attack (computational effort).

Table 5: Known-plaintext attack: success rate and effort (m = 12, rails 2-20, 200 trials per row)

Ciphertext length n	Known k	Strict %	Ranked %	Avg. survivors	Hypotheses tested	Char ops	Time (ms)
50	8	0.0	0.0	19.00	19	1102	0.84
50	12	0.0	99.5	19.00	19	2178	2.98
50	13	43.5	100.0	2.27	19	1360	1.00
50	14	76.0	100.0	1.47	19	1340	0.86
50	16	90.0	100.0	1.10	19	1359	0.91
50	24	100.0	100.0	1.00	19	1506	1.03
100	8	0.0	0.0	19.00	19	2052	1.68
100	12	0.0	100.0	19.00	19	4128	4.29
100	13	48.5	100.0	1.66	19	2413	1.36
100	14	97.0	100.0	1.03	19	2369	1.33
100	16	100.0	100.0	1.00	19	2404	1.44
100	24	100.0	100.0	1.00	19	2556	1.69
500	8	0.0	0.0	19.00	19	9652	3.42
500	12	0.0	100.0	19.00	19	19728	16.06
500	13	44.5	100.0	1.84	19	11170	5.17
500	14	98.5	100.0	1.01	19	10774	4.79
500	16	100.0	100.0	1.00	19	10804	4.74
500	24	100.0	100.0	1.00	19	10956	5.46
1000	8	0.0	0.0	19.00	19	19152	6.08
1000	12	0.0	100.0	19.00	19	39228	29.58
1000	13	48.0	100.0	1.73	19	21972	9.77
1000	14	97.0	100.0	1.03	19	21296	9.95
1000	16	100.0	100.0	1.00	19	21304	8.97
1000	24	100.0	100.0	1.00	19	21456	9.50

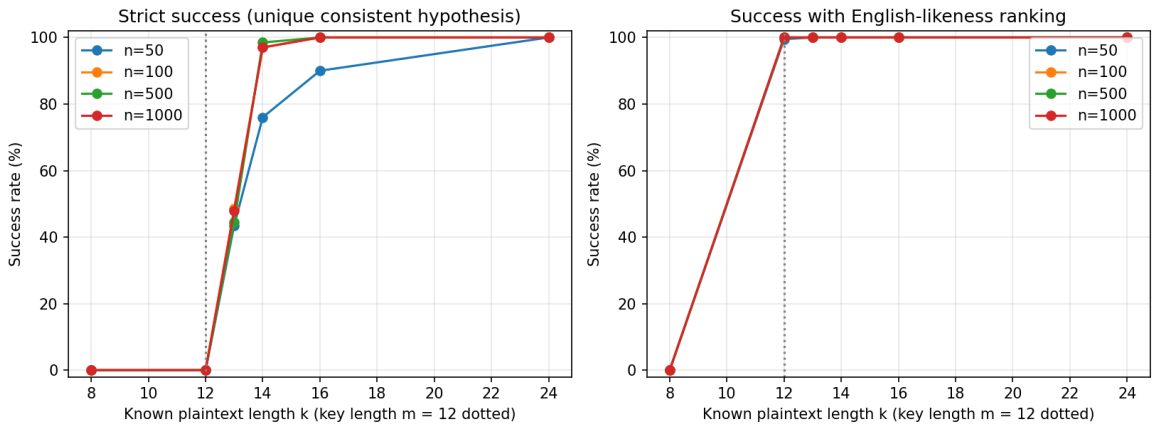


Figure 7: Known-plaintext attack success rate against the known-plaintext length k

Interpretation.

- **k < m (k = 8): 0% success.** The key cannot be fully recovered; all 19 rail counts survive and the attack reports *insufficient known plaintext* instead of guessing.
- **k = m = 12: strict 0%.** With one equation per key slot there is no redundancy, so every rail count is consistent. The English-likeness ranking still picks the right answer in 99.5 to 100% of trials, but only because the plaintext is English.
- **k = 13:** about 1.7 to 1.8 hypotheses survive for n >= 100 (2.27 at n = 50), matching the prediction 1 + 18/26 = 1.69; strict success is 44 to 49%.
- **k >= 14 with n >= 100:** strict success 97 to 100%, and with k = 24 the attack succeeded in all 800 of 800 trials. The average of 1.03 survivors at k = 14 matches 1 + 18/676 = 1.027.
- **Short ciphertexts (n = 50)** need slightly more known text for the strict test (76% at k = 14) because for tiny n different rail counts share more of their first positions.
- **Computational effort** grows linearly with the ciphertext length: about 19 x n operations (about 21,300 operations at n = 1000 and k = 16, around 9 to 10 ms in Python) and is almost independent of k once k > m. Exhaustive key search would need up to 26^{12} (about 9.5 x 10^{16}) keywords.

Table 6: Ciphertext-only attack: success rate against ciphertext length (40 trials per row, m = 12)

Ciphertext length n	Trials	Full recovery %	Char ops	Time (ms)
100	40	0.0	157928	74.2
250	40	65.0	163628	72.4
500	40	97.5	173128	71.1
1000	40	100.0	192128	74.8
2000	40	100.0	230128	87.7
4000	40	100.0	306128	111.9

HCC therefore also falls without any known plaintext once roughly 500 or more letters of English are available (97.5% at n = 500, 100% from n = 1000, but 0% at n = 100). *Limitation:* the test plaintexts come from a small English corpus, so they are English-like but somewhat repetitive; real prose would be a little harder at the shortest lengths.

6. Security Analysis

6.1 Benefits of Combining Substitution and Transposition

Substitution hides which letters occur; transposition hides where they occur. Vigenere flattens letter frequencies, and Rail Fence removes the regular spacing of letters that share a key letter, so neither the plain histogram attack nor the standard Kasiski/IC key-length analysis can be applied directly. Mixing a substitution with a transposition is also the principle behind modern ciphers, which repeat such rounds many times with key-dependent operations.

6.2 Strengths and Weaknesses of the Individual Stages

Table 7: Strengths and weaknesses of the two stages

Cipher	Strengths	Weaknesses

Cipher	Strengths	Weaknesses
Vigenere (Stage 1)	Defeats simple frequency analysis; key space 26^m; easy to implement	Repeating key (every m-th letter is a Shift cipher); Kasiski/IC reveal m; key follows from known plaintext by subtraction
Rail Fence (Stage 2)	Hides position structure; cheap; reversible	Only about n - 1 keys; no secret mixing; keeps letter frequencies; one rail guess is easy to test

6.3 The Effect of Using Both Together

The combined key space (about 2^{60}) is large, but security is **not the product of the two key spaces**. The stages are separable: the known-plaintext attack searches the rail count (about n options) and then reads the keyword off by subtraction, never touching the 26^{12} keyword space. The complexity of the attack is O(R x n), not O(26^{12} x R).

6.4 Vulnerability to Known-Plaintext Attacks

Demonstrated: the complete key is recovered in milliseconds once more than m letters of the plaintext are known. This is realistic, because real messages often begin with predictable text (greetings, headers, standard phrases). A known segment of m letters only decides the key after the language model is used, and fewer than m letters leave part of the key unknown.

6.5 Vulnerability to Frequency Analysis

A naive histogram comparison fails (IC 0.040 and no consistent shift), but a column-wise attack after a search over rail counts succeeds with about 500 letters (Section 5.6). The flattening of letter frequencies is therefore a weak defence, not a proof of security.

6.6 Why HCC Is Educational and Not Suitable for Real Security

- The key stream repeats with period m and there is no diffusion between positions beyond a fixed permutation.
- The transposition key space is tiny and is searched exhaustively in the attack.
- Encryption is deterministic and has no randomization: identical messages give identical ciphertexts.
- There is no integrity or authentication; an attacker can modify the ciphertext undetected.
- The same key is reused for all messages, which makes known-plaintext attacks cumulative.
- Modern systems use vetted algorithms such as AES-GCM or ChaCha20-Poly1305. **We do not claim that HCC is secure against modern cryptanalysis.**

7. Performance Metrics

7.1 Measurement Method

All times are measured with Python's `time.perf_counter()`. Each function is run once as an untimed warm-up and then repeatedly (30 runs for encryption and decryption, 10 runs for the attack); the mean is reported to reduce noise. Plaintexts are English-like with a fixed random

seed; HCC uses the demonstration key `SECURITYKEYS` with 4 rails, and the Shift cipher uses shift 7. The attack uses a known segment of 36 letters (or the whole message if shorter), m = 12 and 19 rail counts (2 to 20). Environment: Windows 11, CPython 3.11.9. Absolute times depend on the machine; the trends do not. The experiment code is `performance.py` and the raw data is in `results/timing.csv`.

7.2 Encryption, Decryption and Attack Time

Table 8: Measured mean execution time in milliseconds for HCC, Shift cipher and the known-plaintext attack

Length n	HCC encrypt	HCC decrypt	Shift encrypt	Shift decrypt	KPA attack	HCC / Shift (encrypt)
50	0.0738	0.1028	0.0315	0.0466	0.9374	2.34x
100	0.1710	0.1293	0.0467	0.0479	1.7378	3.66x
500	0.7966	0.7638	0.4046	0.2800	5.6126	1.97x
1000	1.2931	1.2611	0.5287	0.4612	10.6891	2.45x
5000	6.4131	6.6153	2.6031	2.5871	51.0189	2.46x
10000	12.8603	12.0730	4.8036	4.9255	132.6981	2.68x

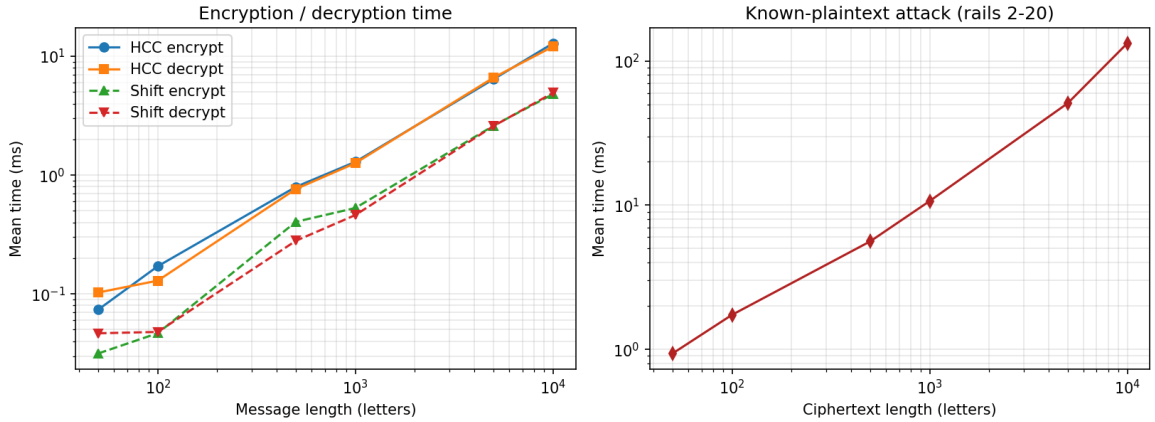


Figure 8: Mean running time against message length (log-log scale): cipher operations (left) and the known-plaintext attack (right)

Effect of message length. When the message length grows 200 times (from 50 to 10000 letters), HCC encryption time grows 174 times. On a log-log graph the curves are straight lines of slope close to 1, which is the signature of linear, O(n), behaviour. The attack time also grows linearly with n, because each of the 19 rail guesses undoes an O(n) permutation. The smallest messages take only tens of microseconds, so those entries are the noisiest, which explains the irregular ratio column.

7.3 Comparison with the Shift Cipher

Table 9: HCC compared with the single-stage Shift cipher

Property	Shift cipher	HCC
Stages	1	2 (substitution + transposition)
Key	Integer 0-25	12-letter keyword + rail count
Key space	26 (about 4.7 bits)	26^12 x about n (about 2^60 for short messages)

Property	Shift cipher	HCC
Time complexity (enc / dec)	O(n) / O(n)	O(n) / O(n)
Measured at n = 1000	0.5287 / 0.4612 ms	1.2931 / 1.2611 ms
Relative cost	1x	about 2 to 3.7x (constant factor)
Frequency analysis	Breaks it with at most 26 trials	Naive histogram matching fails (IC 0.040)
Known-plaintext attack	One known letter reveals the key	Needs k > m letters; about 19 x n operations
Ciphertext-only attack	Trivial	Works with about 500 or more English letters

HCC pays a constant-factor overhead (a second pass plus index bookkeeping) for resistance to the simplest attacks. Both ciphers have the same asymptotic complexity, so the extra security is cheap but also limited.

7.4 Time and Space Complexity of All Major Algorithms

n = message length, m = key length, k = known letters, R = number of rail counts searched, |M| = number of key lengths tried, c = number of surviving hypotheses.

Table 10: Complexity summary

Algorithm	Time	Space	Reason
Normalization	O(n)	O(n)	One pass over the input
Vigenere encrypt / decrypt	O(n)	O(n)	One modular operation per letter
Rail Fence encrypt / decrypt	O(n)	O(n)	Zig-zag pattern and buckets, independent of r
HCC encrypt / decrypt	O(n)	O(n)	Sum of the two stages
Shift cipher	O(n)	O(n)	One operation per letter
Known-plaintext attack	O(R(n + k|M|) + c n)	O(n)	Per rail guess: invert permutation, derive k letters; ranking decrypts survivors
Frequency analysis	O(n + 26)	O(1)	Count letters, compare 26 values
Ciphertext-only attack	O(R(n + 26^2 m))	O(n)	Per rail: split into m columns, try 26 shifts each
Exhaustive key search	O(26^m R n)	O(n)	Infeasible for m = 12, hence the structural attack

8. Suggested Improvements

The following changes raise the cost of the attacks while adding only O(n) work and little code:

Table 11: Improvements, their security benefit and their added cost

Improvement	Security benefit	Added complexity
Keyed columnar transposition instead of Rail Fence	Transposition key space grows from about n to (number of columns)!; the permutation depends on a secret word	One extra sort of the key word; O(n)
Autokey Vigenere (key stream continues with the plaintext)	No repeating key: the consistent-keyword test and Kasiski/IC analysis no longer apply	One line changed; O(n)
Sandwich: Vigenere -> transposition -> Vigenere with a second keyword	Known plaintext is no longer aligned with a single substitution layer, which defeats the consistency test	One more O(n) pass

Improvement	Security benefit	Added complexity
Long random keys (20 letters or more), never a dictionary word, used once per message	Raises the key space and prevents cumulative known-plaintext attacks	None (key management only)
Variable start rail / offset as part of the key	Enlarges the transposition key space at no real cost	A few lines in the zig-zag function

None of these changes makes the cipher secure by modern standards; they illustrate how security can be improved cheaply within the classical framework.

9. Implementation and Source Code

The implementation uses only the Python standard library (matplotlib is optional and is used only for graphs). It is organized into small modules, each with docstrings and input validation.

Table 12: Project structure

File	Purpose
`main.py`	Menu-driven command-line interface (8 options) and a `--demo` mode
`utils.py`	Alphabet handling, normalization, key validation, `ValidationError`
`vigenere.py`, `rail_fence.py`	The two classical stages
`hybrid_cipher.py`	HCC: Stage 1 then Stage 2, and the reverse for decryption
`shift_cipher.py`	Single-stage baseline for the comparison
`attack.py`	Known-plaintext attack, derivation table, ciphertext-only attack
`frequency_analysis.py`	Letter frequencies, chi-squared, index of coincidence, Shift frequency attack
`sample_data.py`	Reproducible English-like test text and random keys
`performance.py`	Timing and attack-success experiments, CSV and PNG output
`tests/`	52 `unittest` tests (round trips, validation, attack success and failure, experiments, CLI)

9.1 Running the Project

```
python main.py                          # interactive menu (options 1-8)
python main.py --demo                   # automatic walk-through of options 1-5 and 7
python -m unittest discover -s tests -v # 52 tests
python performance.py                   # full experiments (about 1 minute), writes
results/
```

9.2 Testing

The test suite verifies textbook vectors for Vigenere and Rail Fence, round trips for every message length from 1 to 79 with rails 2 to 11, short and long messages, messages shorter and longer than the key, invalid keys, empty input, repeated letters, 60 random successful attacks, insufficient or incorrect known plaintext, rail counts outside the search range, and that the recovered keys reproduce the supplied ciphertext. All 52 tests pass.

9.3 Source Code Repository

The complete source code, tests, experiment results and this report are submitted in a GitHub repository:

GitHub repository link: https://github.com/<username>/HCC-Project (insert the final link here)

10. Limitations

- Only the letters A-Z are supported; case, spaces and punctuation are lost by design.
- The attack assumes a known key length (a range is supported at proportional cost), a contiguous known segment and a known position of that segment.
- The experiments use one key length (m = 12), a small built-in English corpus and one machine.
- The language-model tie-break assumes English plaintext.
- Timings of very short messages are at microsecond scale and therefore noisy.

11. Conclusion

HCC satisfies the assignment: two classical techniques in two stages, a 12-letter keyword, arbitrary-length A-Z messages, a working known-plaintext attack that genuinely recovers the keys, measured success rates and computational effort, linear-time encryption and decryption and a quantified comparison with the Shift cipher. The main lesson is that **combining weak ciphers does not multiply their strength**: the stages can be attacked separately, and with more than m known letters the keys are recovered in milliseconds. HCC is a good teaching vehicle and should never be used to protect real data.

References

[1] W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson (chapter on classical encryption techniques).

[2] A. Menezes, P. van Oorschot and S. Vanstone, Handbook of Applied Cryptography, CRC Press, 1996 (classical ciphers, Section 7.3).

[3] D. Kahn, The Codebreakers: The Comprehensive History of Secret Communication, Macmillan, 1967.

[4] R. Lewand, Cryptological Mathematics, Mathematical Association of America, 2000 (English letter frequencies).

[5] W. F. Friedman, The Index of Coincidence and Its Applications in Cryptography, Riverbank Laboratories, 1922.

[6] A. Kerckhoffs, "La cryptographie militaire," Journal des sciences militaires, 1883.

[7] J. Katz and Y. Lindell, Introduction to Modern Cryptography, CRC Press (attack models and Kerckhoffs' principle).

[8] S. Singh, The Code Book, Doubleday, 1999.

[9] Python Software Foundation, Python documentation: time.perf_counter and unittest, https://docs.python.org/3/.